Solución orientativa: prácticas de Git y GitHub

Módulo PI2 · ASIR2

Paco Aldarias

30 de octubre de 2026

Índice

1. Solución orientativa: prácticas de Git y GitHub **1**
1.1. Módulo PI2 · ASIR2 1

2. GitHub Education y primer archivo **1**

3. Rama, documentación y pull request **2**
3.1. Crear el procedimiento 2

4. Crear y resolver un conflicto **3**
4.1. Primera rama 3
4.2. Segunda rama 3
4.3. Integrar y resolver 3

5. Guardar y recuperar cambios con stash **4**

6. Comparar `reset --soft` y `reset --mixed` **4**

1. Solución orientativa: prácticas de Git y GitHub

1.1. Módulo PI2 · ASIR2

Esta guía propone una solución posible para ud5-práctica.md. Sustituye `TU-USUARIO` y `Nombre Apellido` por tus propios datos. Los valores DNS de los ejemplos pertenecen a rangos reservados para documentación: no los configures como servidores reales.

2. GitHub Education y primer archivo

La creación de la cuenta se completa en github.com. Después, desde GitHub Education, se inicia la solicitud de verificación siguiendo las instrucciones que aparezcan en ese momento. GitHub debe evaluar la solicitud: no existe un comando que garantice la aprobación, y el estado de estudiante no se debe fingir.

El repositorio de prácticas puede ser público o privado según indique el profesor. Una estructura mínima sería:

```
ud5-practica/
+-- pacoaldarias.txt
```

El archivo de texto contiene solo el nombre y los apellidos del alumno:

```
Paco Aldarias
```

En GitHub, el archivo se puede crear desde **Add file > Create new file**. Introduce `pacoaldarias.txt`, escribe el contenido y confirma el cambio directamente en `main` con el mensaje:

```
docs: añadir archivo personal de la práctica
```

Para clonar el repositorio en el equipo:

```
git clone https://github.com/TU-USUARIO/ud5-practica.git
cd ud5-practica
git status
```

Si la cuenta aún está pendiente de verificación, se puede continuar con el repositorio y entregar el estado real de la solicitud. No publiques capturas que revelen documentos académicos, información personal o datos de acceso. Los beneficios de Copilot se activan solo si GitHub confirma la elegibilidad y están disponibles para esa cuenta.

3. Rama, documentación y pull request

3.1. Crear el procedimiento

Desde el repositorio clonado, actualiza `main` y crea la rama solicitada:

```
git switch main
git pull --ff-only
git switch -c docs/procedimiento-backup
```

Un ejemplo breve de `procedimiento-backup.md`:

```
# Comprobación de una copia de seguridad de laboratorio

## Objetivo
Comprobar que la copia de prueba se puede leer y restaurar.

## Procedimiento
1. Identificar la copia y la fecha, sin publicar datos sensibles.
2. Verificar el resultado y el tamaño esperado.
3. Restaurar una muestra en un directorio temporal de laboratorio.
4. Comparar los archivos restaurados con la muestra de prueba.
5. Registrar la fecha y el resultado de la comprobación.

## Criterio de aceptación
La muestra se restaura y se puede leer correctamente.
```

Ajusta el procedimiento a la herramienta utilizada. No incluyas contraseñas, tokens ni rutas que revelen información sensible.

Revisa y confirma el cambio:

```
git status
git diff
git add procedimiento-backup.md
git diff --staged
git commit -m "docs: añadir procedimiento de backup"
git push -u origin docs/procedimiento-backup
```

En GitHub, abre una PR desde `docs/procedimiento-backup` hacia `main`. Ejemplo:

- **Título:** `docs: documentar comprobación de copias de seguridad`
- **Descripción:** añade un procedimiento para verificar y restaurar una muestra de una copia de laboratorio.
- **Comprobaciones:**
  - ☒ Revisado el procedimiento.
  - ☒ No se han incluido credenciales ni datos reales.
  - ☒ La restauración descrita se limita a un entorno de laboratorio.

Después de que la PR se integre en GitHub, sincroniza la rama local antes de continuar:

```
git switch main
git pull --ff-only
```

4. Crear y resolver un conflicto

La práctica crea dos ramas desde el mismo `main`, cada una con una versión diferente del mismo archivo nuevo. Eso produce un conflicto de tipo **add/add** al fusionarlas.

4.1. Primera rama

```
git switch main
git pull --ff-only
git switch -c conflicto-a
```

Crea `configuracion-laboratorio.txt`:

```
servidor_dns=192.0.2.10
```

Confirma la primera versión:

```
git add configuracion-laboratorio.txt
git commit -m "docs: documentar DNS del laboratorio"
```

4.2. Segunda rama

Vuelve a la rama base —que aún no contiene el archivo de la primera rama— y crea la rama competidora:

```
git switch main
git switch -c conflicto-b
```

Crea el archivo con el otro valor de ejemplo y confirma el cambio:

```
servidor_dns=192.0.2.53
```

```
git add configuracion-laboratorio.txt
git commit -m "docs: proponer DNS alternativo de laboratorio"
```

4.3. Integrar y resolver

Integra primero `conflicto-a` y luego intenta integrar `conflicto-b`:

```
git switch main
git merge conflicto-a
git merge conflicto-b
git status
```

El segundo `git merge` informa del conflicto y termina con un estado distinto de cero; es esperado en este ejercicio. Git deja el archivo sin resolver hasta que se edite. Puede contener marcadores similares a estos:

```
<<<<<<< HEAD
servidor_dns=192.0.2.10
=======
servidor_dns=192.0.2.53
>>>>>>> conflicto-b
```

Como ejemplo de resolución, se puede acordar conservar el primer valor y dejar el archivo así:

```
servidor_dns=192.0.2.10
```

La elección es una decisión del equipo, no una regla de Git. Comprueba el contexto, elimina todos los marcadores, revisa el resultado y finaliza el merge:

```
git diff
git add configuracion-laboratorio.txt
git commit -m "merge: resolver conflicto de DNS del laboratorio"
git status
```

Al finalizar, `git status` debe indicar que el árbol de trabajo está limpio. El historial debe mostrar los commits de ambas ramas y el commit de merge. `192.0.2.10` y `192.0.2.53` son direcciones reservadas para documentación: usa valores ficticios y no los despliegues en una red real.

5. Guardar y recuperar cambios con stash

En `main`, crea o modifica `notas-laboratorio.md` y no lo confirmes. Por ejemplo, añade una nota de prueba:

```
Comprobar el inventario del laboratorio antes de realizar el backup.
```

Guarda el cambio temporalmente:

```
git status
git stash push -m "notas de laboratorio pendientes"
git stash list
git status
```

El cambio ya no aparece en los archivos de trabajo. Consulta y recupera la primera entrada, sin eliminarla todavía:

```
git stash show -p stash@{0}
git stash apply stash@{0}
git diff
```

Comprueba que la nota se ha recuperado. Después elimina la entrada recuperada:

```
git stash drop stash@{0}
git stash list
```

`git stash apply` aplica el contenido y mantiene la entrada en la lista; `git stash pop` intenta aplicarlo y retirar la entrada. Aplicar primero y eliminar después permite comprobar la recuperación antes de descartar la copia temporal.

6. Comparar `reset --soft` y `reset --mixed`

Usa una rama temporal y un archivo de prueba que no contenga información importante:

```
git switch main
git switch -c prueba-reset
```

Crea `reset-ejemplo.txt` con una línea y confirma el primer estado:

```
version=1
```

```
git add reset-ejemplo.txt
git commit -m "test: crear archivo para probar reset"
```

Modifica la línea y crea el segundo commit:

```
version=2
```

```
git add reset-ejemplo.txt
git commit -m "test: actualizar archivo para probar reset"
git log --oneline -3
git status
```

En este punto, `HEAD` está en el segundo commit y el árbol de trabajo está limpio. Deshaz ese commit con `--soft`:

```
git reset --soft HEAD~1
git status
git diff --staged
```

Ahora `HEAD` apunta al primer commit, pero el cambio de `version=1` a `version=2` sigue preparado en el *staging area*. Confírmalo otra vez para poder hacer la segunda comparación:

```
git commit -m "test: recuperar cambio de prueba"
git status
```

Deshaz el commit recién creado con el modo predeterminado (`--mixed`):

```
git reset HEAD~1
git status
```

```
git diff
git diff --staged
```

El resultado esperado en ambos resets es que la rama retroceda al commit con `version=1` y el contenido del archivo mantenga `version=2`. Cambia el área de preparación:

Operación	`HEAD` después del reset	Área de preparación	Archivo de trabajo
`git reset --soft HEAD~1`	Primer commit (`version=1`)	Conserva `version=2` preparado	Conserva `version=2`
`git reset HEAD~1` (`--mixed`)	Primer commit (`version=1`)	Quita el cambio del *staging area*	Conserva `version=2` como cambio sin preparar

Deja activa la rama `prueba-reset`, revisa `git status` y decide qué hacer con el archivo de prueba. No uses `git reset --hard` ni elimines la rama mientras haya cambios que quieras conservar.